Objetivo: Aprender lo básico de un Sistema de Control de Versiones (SCV).

Antes de la clase: preparar el entorno

La actividad de esta clase se hace con Git funcionando en su computadora. Instalar y configurar toma unos 15 minutos y no se puede improvisar durante la clase.

1. Instalar Git (https://git-scm.com/downloads), GitHub CLI (https://cli.github.com/) y, si todavía no lo tienen, VSCode (https://code.visualstudio.com/). Aceptar las opciones por defecto de los instaladores.
2. Crear una cuenta en GitHub (https://github.com/signup), con un nombre de usuario profesional: va a ser visible en el portfolio de cada uno.
3. Abrir una terminal (en Windows, *Git Bash*) y ejecutar estas cinco líneas, reemplazando el nombre y el correo por los propios:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global core.editor "code --wait"
git config --global pull.rebase false
gh auth login
```

Las dos primeras líneas le dicen a Git quién es uno: ese nombre y ese correo quedan grabados en cada cambio que se guarde. Las dos siguientes evitan problemas conocidos. La última abre el navegador para iniciar sesión en GitHub: hay que responder GitHub.com → HTTPS → Y → Login with a web browser.

4. Verificar que quedó todo bien con git --version y gh auth status. El primero debe devolver un número de versión; el segundo, una línea que diga Logged in to github.com.

Si algo falla, resolverlo antes de la clase o avisar al llegar.

El problema

Es crucial contar con un sistema que guarde un historial de nuestro trabajo. Sin él, versionar es esto:

- tp_final.zip, tp_final_v2.zip, tp_final_v2_corregido.zip.
- *"¿Cuál era la versión que andaba?"*
- *"Toqué algo y dejó de compilar, pero no me acuerdo qué toqué."*
- *"Le pasé el proyecto a mi compañero, los dos lo modificamos, y ahora hay dos proyectos distintos."*

El esquema falla por tres motivos: no hay un historial confiable, no hay forma de volver atrás con precisión y no hay manera ordenada de que dos personas trabajen a la vez sobre lo mismo. Un Sistema de Control de Versiones (SCV) resuelve exactamente esos tres problemas: registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, permite recuperar versiones específicas, comparar cambios, revertir el proyecto a un estado anterior y colaborar entre varios desarrolladores.

En esta materia el SCV no es un tema aislado: es la herramienta con la que se va a construir y entregar el proyecto integrador (ISFPP) durante todo el cuatrimestre.

Git y el repositorio

Git es un SCV distribuido. A diferencia de los sistemas centralizados, cada desarrollador posee una copia completa del proyecto y de todo su historial en su máquina.

Un repositorio es el directorio donde reside el proyecto y que está siendo rastreado por Git. Contiene los archivos del proyecto y el historial completo de los cambios, guardado en una carpeta oculta .git. Trabajamos permanentemente con dos:

- **Repositorio local**: la copia que está en la propia computadora. Acá se realizan y se guardan los cambios.
- **Repositorio remoto**: una versión alojada en un servidor, como los que provee GitHub. Es el punto de encuentro donde el equipo sincroniza su trabajo.

El flujo de trabajo consiste en sincronizar estos dos repositorios.

GitHub no son lo mismo: Git es el programa que gestiona el historial, está instalado en la máquina y funciona sin conexión a internet. GitHub es un servicio que aloja repositorios Git en la nube y agrega funciones de colaboración. Existen alternativas equivalentes, como GitLab o Bitbucket.

Cómo se guarda un cambio

Guardar un cambio en Git es un proceso de dos pasos, no es un simple “Guardar como…”. Un archivo modificado atraviesa tres lugares:

El directorio de trabajo son los archivos tal como están en el disco, con los cambios recién hechos. Git los ve, pero todavía no los guardó.

El área de preparación (*staging*) es un lugar intermedio donde se indica qué cambios van a formar parte de la próxima “fotografía” del proyecto. La acción de poner cambios ahí se llama git add. Este paso intermedio existe para no estar obligado a guardar todo junto: se puede confirmar el trabajo terminado y dejar afuera un archivo de prueba a medio hacer.

El repositorio local es donde vive el historial. La acción de tomar todo lo que está en el área de preparación y guardarlo como una versión permanente se llama git commit, y el resultado es un **commit**: una fotografía del estado del proyecto en un momento significativo. Cada commit tiene un identificador único, el autor, la fecha y, fundamentalmente, un mensaje que describe qué se cambió y por qué.

Ese mensaje no es un trámite: es lo que va a leer el equipo, y uno mismo dentro de tres semanas, para entender el historial. Un buen mensaje describe qué cambió, en una línea corta y en modo imperativo: Agrega clase Estudiante con atributos legajo y nombre, Corrige cálculo del promedio en Materia. Mensajes como cambios, asdasd o final final ahora si no le dicen nada a nadie. Del mismo modo, un commit debería representar una unidad de trabajo con sentido propio: no un archivo suelto a medio escribir, ni tres funcionalidades distintas mezcladas.

Sincronización: clone, push y pull

Una vez hechos uno o más commits en el repositorio local, esos cambios solo existen en esa máquina. Para que existan para el equipo hay que sincronizar:

- **Clone**: descargar por primera vez un repositorio remoto completo, con todo su historial. Se hace una sola vez por proyecto y por computadora.

- **Push** (empujar): enviar los commits locales al repositorio remoto. Así se comparte el trabajo y se respalda en la nube.
- **Pull** (tirar): traer los commits que otros subieron al remoto y fusionarlos con el repositorio local, para trabajar sobre la versión más actualizada.

De ahí sale la regla práctica de cualquier equipo: **pull antes de empezar a trabajar, push cuando se termina**.

rama (*branch*)
permite llevar adelante varias líneas de desarrollo en paralelo. La rama principal, llamada main, contiene el código estable y funcional.
Cuando se quiere trabajar en algo nuevo (una funcionalidad, la corrección de un error, un experimento) se crea una rama nueva, que es una copia del proyecto en ese momento. Se puede trabajar y hacer todos los commits que se quieran en ella con la seguridad de que main no se ve afectada. Cuando el trabajo está terminado y probado, la rama se **fusiona** (*merge*) de vuelta con main, integrando los cambios al proyecto principal.

Conflicto
Ocurre cuando se fusionan dos líneas de trabajo que modificaron la misma línea del mismo archivo. Git no sabe cuál de los dos cambios es el correcto.
Es importante ver también qué no es un conflicto: si dos personas modifican archivos distintos, o líneas distintas del mismo archivo, Git fusiona todo automáticamente y no hay que hacer nada.
Ante la ambigüedad, en lugar de adivinar, Git detiene la fusión y marca el archivo así:

```
public class App {
    public static void main(String[] args) {
<<<<<<< HEAD
        System.out.println("Hola, POO 2026!");
=======
        System.out.println("Buenas, mundo!");
>>>>>>> main
    }
}
```

Entre <<<<<<< HEAD y ======= está la versión local; entre ======= y >>>>>>>, la que llegó del remoto. La resolución es siempre manual: la persona edita el archivo y decide qué conservar (una versión, la otra, o una combinación de las dos), borra los tres marcadores y confirma un nuevo commit que cierra la fusión.

.**gitignore**
No todos los archivos de un proyecto deben guardarse en el historial. Los archivos compilados (.class en Java), la configuración del IDE, las contraseñas y las dependencias descargadas no pertenecen al repositorio: se regeneran solos, cambian todo el tiempo, ensucian cada commit y son una fuente permanente de conflictos inútiles.

El archivo .gitignore es un archivo de texto ubicado en la raíz del proyecto, donde se listan los archivos y carpetas que Git debe ignorar por completo. Suelen estar predefinidos según el lenguaje o framework; GitHub ofrece una plantilla para Java que incluye, entre otras reglas, *.class, bin/ y target/.

El flujo, en resumen

1. **Clonar** el repositorio del equipo (una sola vez).
2. **Pull** antes de empezar a trabajar.
3. Programar, creando o modificando archivos.
4. **Preparar** los cambios que se quieren guardar (*stage*).
5. **Confirmar** esos cambios en el repositorio local (*commit*), con un mensaje claro.
6. **Push** para sincronizar el trabajo con el repositorio remoto.
7. Usar **ramas** para mantener el orden y experimentar de forma segura.

En clase vamos a recorrer este flujo desde la línea de comandos, que es donde cada uno de estos conceptos tiene un nombre propio y visible. Existen herramientas visuales: GitHub Desktop y el panel de control de versiones de VSCode que ejecutan exactamente los mismos comandos detrás de sus botones.

Bibliografía y recursos sugeridos

Para la gestión de versiones con Git, un tema práctico, se recomienda la consulta de recursos especializados. La bibliografía principal de la cátedra se centra en el paradigma de programación orientada a objetos, por lo que para este tema sugerimos el siguiente material de referencia:

Libro principal:

- Chacon, S., & Straub, B. (2014). *Pro Git*. Apress. Disponible en: https://git-scm.com/book/es/v2. Los capítulos 1 y 2 alcanzan para esta clase.

Recursos adicionales:

- Documentación oficial de Git: https://git-scm.com/doc
- Guías de GitHub: https://docs.github.com/es/get-started
- Visualizador interactivo para practicar ramas y fusiones: https://learngitbranching.js.org/?locale=es_AR